

MYSQL STORED PROCEDURES

Examples and Practice Exercises for MySQL Workbench

2520030395

MONISH PUTTI

1. Setup: Employees Table

```
CREATE DATABASE IF NOT EXISTS employeedb1;  
USE employeedb1;
```

```
CREATE TABLE IF NOT EXISTS employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary DECIMAL(10,2)  
);
```

```
INSERT INTO employees VALUES  
(101,'Rahul',1,50000),  
(102,'Priya',2,65000),  
(103,'Anil',1,55000),  
(104,'Sneha',3,70000),  
(105,'Kiran',2,48000);
```

Query 1 x

Limit to 1000 rows

```

1 • CREATE DATABASE IF NOT EXISTS employeeedb1;
2
3 • USE employeeedb1;
4
5 • CREATE TABLE IF NOT EXISTS employees (
6     emp_id INT PRIMARY KEY,
7     emp_name VARCHAR(50)

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00
NULL	NULL	NULL	NULL

employees 1 x

Output

Action Output

#	Time	Action	Message
✓ 1	11:53:50	CREATE DATABASE IF NOT EXISTS employeeedb1	1 row(s) affected
✓ 2	11:53:50	USE employeeedb1	0 row(s) affected
✓ 3	11:53:50	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), d...	0 row(s) affected
✓ 4	11:53:50	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sneh...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
✓ 5	11:54:14	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned

2. Verify Data

Question: Display all employee records.

SELECT * FROM employees;

Query 1 x SQLAdditions

Limit to 1000 rows

1 • `SELECT * FROM employees;`

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	51000.00
	102	Priya	3	67000.00
	103	Anil	1	57000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
*	NULL	NULL	NULL	NULL

Form Editor

employees 23 x Apply Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 78	12:19:56	SELECT @salary AS UpdatedSalary LIMIT 0, 1000	1 row(s) returned
⚠ 79	12:20:38	DROP PROCEDURE IF EXISTS TransferSalary	0 row(s) affected, 1 warning(s): 1305 PR
✓ 80	12:20:38	CREATE PROCEDURE TransferSalary(IN fro...	0 row(s) affected
✓ 81	12:20:38	CALL TransferSalary(101,103,2000)	0 row(s) affected
✓ 82	12:20:38	SELECT emp_id, emp_name, salary FROM emplo...	2 row(s) returned
✓ 83	12:22:27	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned

Procedure 1 – Display All Employees

Question: Create a procedure that displays all employees.

```
DROP PROCEDURE IF EXISTS GetEmployees;
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployees()
```

```
BEGIN
    SELECT * FROM employees;
END $$
```

```
DELIMITER ;
```

```
CALL GetEmployees();
```

The screenshot shows a SQL IDE interface. The query editor contains the following SQL code:

```
1 DROP PROCEDURE IF EXISTS GetEmployees;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE GetEmployees()
6 BEGIN
7     SELECT * FROM employees;
8 END $$
9
10 DELIMITER ;
11
```

The result grid displays the following data:

emp_id	emp_name	dept_id	salary
101	Rahul	1	50000.00
102	Priya	2	65000.00
103	Anil	1	55000.00
104	Sneha	3	70000.00
105	Kiran	2	48000.00

The output pane shows the execution log:

#	Time	Action	Message
3	11:53:50	CREATE TABLE IF NOT EXISTS employees (emp_id INT PRIMARY KEY, emp_name VARCHAR(50), ...	0 row(s) affected
4	11:53:50	INSERT INTO employees VALUES (101,'Rahul',1,50000), (102,'Priya',2,65000), (103,'Anil',1,55000), (104,'Sne...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0
5	11:54:14	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned
6	11:57:14	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected, 1 warning(s): 1305 PROCEDURE employeesdb1.GetEmployees does not exist
7	11:57:14	CREATE PROCEDURE GetEmployees() BEGIN SELECT * FROM employees; END	0 row(s) affected
8	11:57:14	CALL GetEmployees()	5 row(s) returned

Procedure 2 – IN Parameter

Question: Create a procedure that accepts an employee ID and displays that employee's details.

```
DROP PROCEDURE IF EXISTS GetEmployeeById;
DELIMITER $$
```

```
CREATE PROCEDURE GetEmployeeById(IN eid INT)
BEGIN
    SELECT * FROM employees
    WHERE emp_id = eid;
END $$
```

```
DELIMITER ;
```

CALL GetEmployeeById(102);

The screenshot displays a SQL IDE interface. The top pane, titled 'Query 1', contains the following SQL code:

```
1 • DROP PROCEDURE IF EXISTS GetEmployeeById;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE GetEmployeeById(IN eid INT)
6 BEGIN
7     SELECT *
8     FROM employees WHERE emp_id = eid;
9 END $$
10
```

The bottom-left pane shows the 'Result Grid' with one row of data:

emp_id	emp_name	dept_id	salary
102	Priya	2	65000.00

The bottom-right pane shows the 'Output' window with the 'Action Output' tab selected. It contains a log of database actions:

#	Time	Action	Message
6	11:57:14	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected, 1 warning(s): 1305 PR
7	11:57:14	CREATE PROCEDURE GetEmployees() BEGIN ...	0 row(s) affected
8	11:57:14	CALL GetEmployees()	5 row(s) returned
9	12:00:22	DROP PROCEDURE IF EXISTS GetEmployeeById	0 row(s) affected, 1 warning(s): 1305 PR
10	12:00:22	CREATE PROCEDURE GetEmployeeById(IN eid ...	0 row(s) affected
11	12:00:22	CALL GetEmployeeById(102)	1 row(s) returned

Procedure 3 – Update Salary

Question: Create a procedure that accepts employee ID and amount, then increases the employee's salary.

```
DROP PROCEDURE IF EXISTS IncreaseSalary;  
DELIMITER $$
```

```
CREATE PROCEDURE IncreaseSalary(  
    IN eid INT,  
    IN amount DECIMAL(10,2)  
)
```

```
BEGIN  
    UPDATE employees  
    SET salary = salary + amount  
    WHERE emp_id = eid;  
END $$
```

```
DELIMITER ;
```

```
CALL IncreaseSalary(101,5000);  
SELECT * FROM employees WHERE emp_id=101;
```

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS IncreaseSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE IncreaseSalary(
6     IN eid INT,
7     IN amount DECIMAL(10,2)
8 )
9 BEGIN
10    UPDATE employees

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	55000.00
NULL	NULL	NULL	NULL

Form Editor

employees 4 x Apply Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 10	12:00:22	CREATE PROCEDURE GetEmployeeById(IN eid ...	0 row(s) affected
✓ 11	12:00:22	CALL GetEmployeeById(102)	1 row(s) returned
⚠ 12	12:01:35	DROP PROCEDURE IF EXISTS IncreaseSalary	0 row(s) affected, 1 warning(s): 1305 PR
✓ 13	12:01:35	CREATE PROCEDURE IncreaseSalary(IN eid ...	0 row(s) affected
✓ 14	12:01:35	CALL IncreaseSalary(101,5000)	1 row(s) affected
✓ 15	12:01:35	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned

Procedure 4 – Aggregate Function

Question: Create a procedure to calculate the average salary of all employees.

```

DROP PROCEDURE IF EXISTS AvgSalary;
DELIMITER $$

```

```

CREATE PROCEDURE AvgSalary()

```

```
BEGIN
  SELECT AVG(salary) AS AverageSalary
  FROM employees;
END $$
```

```
DELIMITER ;
```

```
CALL AvgSalary();
```

Query 1 x

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS AvgSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE AvgSalary()
6 BEGIN
7     SELECT AVG(salary) AS AverageSalary
8     FROM employees;
9 END $$
10

```

Result Grid

AverageSalary
58600.000000

Result Grid

Form Editor

Result 5 x

Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 13	12:01:35	CREATE PROCEDURE IncreaseSalary(IN eid ...	0 row(s) affected
✓ 14	12:01:35	CALL IncreaseSalary(101,5000)	1 row(s) affected
✓ 15	12:01:35	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
⚠ 16	12:03:50	DROP PROCEDURE IF EXISTS AvgSalary	0 row(s) affected, 1 warning(s): 1305 PR
✓ 17	12:03:50	CREATE PROCEDURE AvgSalary() BEGIN SE...	0 row(s) affected
✓ 18	12:03:50	CALL AvgSalary()	1 row(s) returned

Procedure 5 – JOIN

Question: Create a procedure to display employee name, department name and salary. Run the department setup first.

```

CREATE TABLE IF NOT EXISTS departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)

```

);

INSERT IGNORE INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'EEE');

DROP PROCEDURE IF EXISTS EmployeeDepartment;
DELIMITER \$\$

CREATE PROCEDURE EmployeeDepartment()
BEGIN
 SELECT e.emp_name, d.dept_name, e.salary
 FROM employees e
 JOIN departments d ON e.dept_id=d.dept_id;
END \$\$

DELIMITER ;

CALL EmployeeDepartment();

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 CREATE TABLE IF NOT EXISTS departments (
2     dept_id INT PRIMARY KEY,
3     dept_name VARCHAR(50)
4 );
5
6 INSERT IGNORE INTO departments VALUES
7 (1, 'CSE'),
8 (2, 'ECE'),
9 (3, 'EEE');

```

Automatically disabled. manually get caret position auto

Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 15	12:01:35	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
⚠ 16	12:03:50	DROP PROCEDURE IF EXISTS AvgSalary	0 row(s) affected, 1 warning(s): 1305 PR
✓ 17	12:03:50	CREATE PROCEDURE AvgSalary() BEGIN SE...	0 row(s) affected
✓ 18	12:03:50	CALL AvgSalary()	1 row(s) returned
✓ 19	12:05:13	CREATE TABLE IF NOT EXISTS departments (...	0 row(s) affected
✓ 20	12:05:13	INSERT IGNORE INTO departments VALUES (1,...	3 row(s) affected Records: 3 Duplicates

Procedure 6 – GROUP BY and AVG

Question: Create a procedure to display department-wise employee count and average salary.

```

DROP PROCEDURE IF EXISTS DepartmentAverageSalary;
DELIMITER $$

```

```

CREATE PROCEDURE DepartmentAverageSalary()

```

```
BEGIN
  SELECT d.dept_name,
         COUNT(e.emp_id) AS EmployeeCount,
         AVG(e.salary) AS AverageSalary
  FROM employees e
  JOIN departments d ON e.dept_id=d.dept_id
  GROUP BY d.dept_id, d.dept_name;
END $$
```

```
DELIMITER ;
```

```
CALL DepartmentAverageSalary();
```

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS DepartmentAverageSalary
2
3 DELIMITER $$
4
5 CREATE PROCEDURE DepartmentAverageSalary()
6 BEGIN
7     SELECT d.dept_name,
8           COUNT(e.emp_id) AS EmployeeCount,
9           AVG(e.salary) AS AverageSalary
10    FROM employees e

```

Result Grid

	dept_name	EmployeeCount	AverageSalary
▶	CSE	2	55000.000000
	ECE	2	56500.000000
	EEE	1	70000.000000

Result 7 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
23	12:06:50	DROP PROCEDURE IF EXISTS EmployeeDepart...	0 row(s) affected, 1 warning(s): 1305 PR
24	12:06:50	CREATE PROCEDURE EmployeeDepartment() B...	0 row(s) affected
25	12:06:50	CALL EmployeeDepartment()	5 row(s) returned
26	12:07:57	DROP PROCEDURE IF EXISTS DepartmentAver...	0 row(s) affected, 1 warning(s): 1305 PR
27	12:07:57	CREATE PROCEDURE DepartmentAverageSalar...	0 row(s) affected
28	12:07:57	CALL DepartmentAverageSalary()	3 row(s) returned

Procedure 7 – IF...ELSE

Question: Display High Salary if salary is at least 60000; otherwise display Low Salary.

```

DROP PROCEDURE IF EXISTS SalaryStatus;
DELIMITER $$

```

```

CREATE PROCEDURE SalaryStatus(IN eid INT)

```

```
BEGIN
  DECLARE sal DECIMAL(10,2);

  SELECT salary INTO sal
  FROM employees
  WHERE emp_id=eid;

  IF sal >= 60000 THEN
    SELECT 'High Salary' AS Status;
  ELSE
    SELECT 'Low Salary' AS Status;
  END IF;
END $$

DELIMITER ;

CALL SalaryStatus(102);
```

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 • DROP PROCEDURE IF EXISTS SalaryStatus;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE SalaryStatus(IN eid INT)
6 BEGIN
7     DECLARE sal DECIMAL(10,2);
8
9     SELECT salary INTO sal
10    FROM employees

```

Result Grid

Status
High Salary

Result Grid

Form Editor

Result 8 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
26	12:07:57	DROP PROCEDURE IF EXISTS DepartmentAver...	0 row(s) affected, 1 warning(s): 1305 PR
27	12:07:57	CREATE PROCEDURE DepartmentAverageSalar...	0 row(s) affected
28	12:07:57	CALL DepartmentAverageSalary()	3 row(s) returned
29	12:08:59	DROP PROCEDURE IF EXISTS SalaryStatus	0 row(s) affected, 1 warning(s): 1305 PR
30	12:08:59	CREATE PROCEDURE SalaryStatus(IN eid INT) ...	0 row(s) affected
31	12:08:59	CALL SalaryStatus(102)	1 row(s) returned

Procedure 8 – OUT Parameter

Question: Return the total number of employees using an OUT parameter.

```

DROP PROCEDURE IF EXISTS EmployeeCount;
DELIMITER $$

```

```

CREATE PROCEDURE EmployeeCount(OUT total INT)

```

```
BEGIN
  SELECT COUNT(*) INTO total
  FROM employees;
END $$

DELIMITER ;

CALL EmployeeCount(@total);
SELECT @total AS TotalEmployees;
```

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS EmployeeCount;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE EmployeeCount(OUT total INT)
6 BEGIN
7     SELECT COUNT(*) INTO total
8     FROM employees;
9 END $$
10

```

Result Grid

TotalEmployees
5

Result 9 x Read Only Context Help Sn

Output

Action Output

#	Time	Action	Message
✓ 30	12:08:59	CREATE PROCEDURE SalaryStatus(IN eid INT) ...	0 row(s) affected
✓ 31	12:08:59	CALL SalaryStatus(102)	1 row(s) returned
⚠ 32	12:10:04	DROP PROCEDURE IF EXISTS EmployeeCount	0 row(s) affected, 1 warning(s): 1305 P
✓ 33	12:10:04	CREATE PROCEDURE EmployeeCount(OUT tot...	0 row(s) affected
✓ 34	12:10:04	CALL EmployeeCount(@total)	1 row(s) affected
✓ 35	12:10:04	SELECT @total AS TotalEmployees LIMIT 0, 1000	1 row(s) returned

Procedure 9 – INOUT Parameter

Question: Receive a bonus through an INOUT parameter and add 5000 to it.

```

DROP PROCEDURE IF EXISTS AddBonus;
DELIMITER $$

```

```

CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))

```

BEGIN

SET bonus = bonus + 5000;
END \$\$

DELIMITER ;

SET @bonus=10000;
CALL AddBonus(@bonus);
SELECT @bonus AS UpdatedBonus;

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS AddBonus;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE AddBonus(INOUT bonus DECIMAL(10,2))
6 BEGIN
7     SET bonus = bonus + 5000;
8 END $$
9
10 DELIMITER ;

```

Result Grid

UpdatedBonus
15000.00

Result Grid
Form Editor

Result 10 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
35	12:10:04	SELECT @total AS TotalEmployees LIMIT 0, 1000	1 row(s) returned
36	12:10:44	DROP PROCEDURE IF EXISTS AddBonus	0 row(s) affected, 1 warning(s): 1305 PR
37	12:10:44	CREATE PROCEDURE AddBonus(INOUT bonus...	0 row(s) affected
38	12:10:44	SET @bonus = 10000	0 row(s) affected
39	12:10:44	CALL AddBonus(@bonus)	0 row(s) affected
40	12:10:44	SELECT @bonus AS UpdatedBonus LIMIT 0, 1000	1 row(s) returned

Procedure 10 – Transaction

Question: Transfer an amount from one employee salary record to another using START TRANSACTION and COMMIT.

```

DROP PROCEDURE IF EXISTS TransferSalaryAmount;
DELIMITER $$

```

```
CREATE PROCEDURE TransferSalaryAmount(  
    IN fromEmp INT,  
    IN toEmp INT,  
    IN amount DECIMAL(10,2)  
)  
BEGIN  
    START TRANSACTION;  
  
    UPDATE employees  
    SET salary=salary-amount  
    WHERE emp_id=fromEmp;  
  
    UPDATE employees  
    SET salary=salary+amount  
    WHERE emp_id=toEmp;  
  
    COMMIT;  
END $$  
  
DELIMITER ;  
  
CALL TransferSalaryAmount(101,102,2000);  
SELECT * FROM employees WHERE emp_id IN (101,102);
```

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS TransferSalaryAmount;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE TransferSalaryAmount(
6     IN fromEmp INT,
7     IN toEmp INT,
8     IN amount DECIMAL(10,2)
9 )
10 BEGIN

```

Result Grid

emp_id	emp_name	dept_id	salary
101	Rahul	1	53000.00
102	Priya	2	67000.00
NULL	NULL	NULL	NULL

employees 11 x Apply Context Help Snip

Output

Action Output

#	Time	Action	Message
39	12:10:44	CALL AddBonus(@bonus)	0 row(s) affected
40	12:10:44	SELECT @bonus AS UpdatedBonus LIMIT 0, 1000	1 row(s) returned
41	12:11:28	DROP PROCEDURE IF EXISTS TransferSalaryA...	0 row(s) affected, 1 warning(s): 1305 PR
42	12:11:28	CREATE PROCEDURE TransferSalaryAmount(...	0 row(s) affected
43	12:11:28	CALL TransferSalaryAmount(101,102,2000)	0 row(s) affected
44	12:11:28	SELECT * FROM employees WHERE emp_id IN (...	2 row(s) returned

4. Useful Procedure Commands

SHOW PROCEDURE STATUS

WHERE Db='employeeedb1';

SHOW CREATE PROCEDURE GetEmployees;

```
DROP PROCEDURE IF EXISTS GetEmployees;
```

5. Why DELIMITER is Used

DELIMITER temporarily changes the statement terminator in MySQL Workbench. It prevents the semicolons inside BEGIN...END from being interpreted as the end of the entire CREATE PROCEDURE statement.

```
DELIMITER $$  
CREATE PROCEDURE Example()  
BEGIN  
    SELECT * FROM employees;  
END $$  
DELIMITER ;
```

6. Student Practice Tasks

1. Create a procedure to display employees whose salary is greater than a given amount.

Query 1 x

Search table data for text in objects selected in the sidebar schema tree

Limit to 1000 rows

```

1 • DROP PROCEDURE IF EXISTS EmployeesAboveSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE EmployeesAboveSalary(IN min_sa
6 BEGIN
7     SELECT *
8     FROM employees
9     WHERE salary > min_salary;
10 END $$

```

Result Grid

	emp_id	emp_name	dept_id	salary
▶	102	Priya	2	67000.00
	104	Sneha	3	70000.00

Export: Wrap Cell

Result Grid

Form Editor

Result 12 x

Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 42	12:11:28	CREATE PROCEDURE TransferSalaryAmount(...	0 row(s) affected
✓ 43	12:11:28	CALL TransferSalaryAmount(101,102,2000)	0 row(s) affected
✓ 44	12:11:28	SELECT * FROM employees WHERE emp_id IN (...	2 row(s) returned
⚠ 45	12:13:52	DROP PROCEDURE IF EXISTS EmployeesAbov...	0 row(s) affected, 1 warning(s): 1305 PR
✓ 46	12:13:52	CREATE PROCEDURE EmployeesAboveSalary(l...	0 row(s) affected
✓ 47	12:13:53	CALL EmployeesAboveSalary(55000)	2 row(s) returned

2. Create a procedure to update an employee's department using IN parameters.

Query 1 x

SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS UpdateEmployeeDepartme
2
3 DELIMITER $$
4
5 CREATE PROCEDURE UpdateEmployeeDepartment(
6     IN eid INT,
7     IN new_dept INT
8 )
9 BEGIN
10     UPDATE employees

```

Result Grid

	emp_id	emp_name	dept_id	salary
▶	102	Priya	3	67000.00
*	NULL	NULL	NULL	NULL

Form Editor

employees 13 x

Apply

Context Help

Output

Action Output

#	Time	Action	Message
✓ 46	12:13:52	CREATE PROCEDURE EmployeesAboveSalary(l...	0 row(s) affected
✓ 47	12:13:53	CALL EmployeesAboveSalary(55000)	2 row(s) returned
⚠ 48	12:14:58	DROP PROCEDURE IF EXISTS UpdateEmployee...	0 row(s) affected, 1 warning(s): 1305 PR
✓ 49	12:14:58	CREATE PROCEDURE UpdateEmployeeDepart...	0 row(s) affected
✓ 50	12:14:58	CALL UpdateEmployeeDepartment(102,3)	1 row(s) affected
✓ 51	12:14:58	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned

3. Create a procedure to return the maximum salary using an OUT parameter.

Query 1 x

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS MaxSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE MaxSalary(OUT max_sal DECIMAL(
6 BEGIN
7     SELECT MAX(salary) INTO max_sal
8     FROM employees;
9 END $$
10

```

Result Grid

MaximumSalary
70000.00

Result 14 x

Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 50	12:14:58	CALL UpdateEmployeeDepartment(102,3)	1 row(s) affected
✓ 51	12:14:58	SELECT * FROM employees WHERE emp_id = 1...	1 row(s) returned
⚠ 52	12:15:54	DROP PROCEDURE IF EXISTS MaxSalary	0 row(s) affected, 1 warning(s): 1305 PR
✓ 53	12:15:54	CREATE PROCEDURE MaxSalary(OUT max_sal ...	0 row(s) affected
✓ 54	12:15:54	CALL MaxSalary(@max_salary)	1 row(s) affected
✓ 55	12:15:54	SELECT @max_salary AS MaximumSalary LIMIT ...	1 row(s) returned

4. Create a procedure to return the employee count of a specified department.

Query 1 x

Limit to 1000 rows

```

5 CREATE PROCEDURE DepartmentEmployeeCount(IN dept
6 BEGIN
7     SELECT COUNT(*) AS EmployeeCount
8     FROM employees
9     WHERE dept_id = dept;
10 END $$
11
12 DELIMITER ;
13
14 CALL DepartmentEmployeeCount(2);

```

Result Grid

	EmployeeCount
1	1

Result 16 x

Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
56	12:16:28	DROP PROCEDURE IF EXISTS DepartmentEmpl...	0 row(s) affected, 1 warning(s): 1305 PR
57	12:16:28	CREATE PROCEDURE DepartmentEmployeeCo...	0 row(s) affected
58	12:16:28	CALL DepartmentEmployeeCount(2)	1 row(s) returned
59	12:16:33	DROP PROCEDURE IF EXISTS DepartmentEmpl...	0 row(s) affected
60	12:16:33	CREATE PROCEDURE DepartmentEmployeeCo...	0 row(s) affected
61	12:16:33	CALL DepartmentEmployeeCount(2)	1 row(s) returned

5. Create a procedure using JOIN to display employee and department details.

Query 1 x

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS EmployeeDepartmentData
2
3 DELIMITER $$
4
5 CREATE PROCEDURE EmployeeDepartmentDetails()
6 BEGIN
7     SELECT
8         e.emp_id,
9         e.emp_name,
10        d.dept_name,

```

Result Grid

	emp_id	emp_name	dept_name	salary
▶	101	Rahul	CSE	53000.00
	102	Priya	EEE	67000.00
	103	Anil	CSE	55000.00
	104	Sneha	EEE	70000.00
	105	Kiran	ECE	48000.00

Result Grid

Form Editor

Result 17 x

Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 59	12:16:33	DROP PROCEDURE IF EXISTS DepartmentEmpl...	0 row(s) affected
✓ 60	12:16:33	CREATE PROCEDURE DepartmentEmployeeCo...	0 row(s) affected
✓ 61	12:16:33	CALL DepartmentEmployeeCount(2)	1 row(s) returned
⚠ 62	12:17:07	DROP PROCEDURE IF EXISTS EmployeeDepart...	0 row(s) affected, 1 warning(s): 1305 PR
✓ 63	12:17:07	CREATE PROCEDURE EmployeeDepartmentDet...	0 row(s) affected
✓ 64	12:17:07	CALL EmployeeDepartmentDetails()	5 row(s) returned

6. Create a procedure to display departments whose average salary is greater than 55000.

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 • DROP PROCEDURE IF EXISTS HighAverageDepartments
2
3 DELIMITER $$
4
5 CREATE PROCEDURE HighAverageDepartments()
6 BEGIN
7     SELECT d.dept_name,
8           AVG(e.salary) AS AverageSalary
9     FROM employees e
10    JOIN departments d

```

Result Grid

	dept_name	AverageSalary
▶	EEE	68500.000000

Result Grid

Form Editor

Result 18 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
62	12:17:07	DROP PROCEDURE IF EXISTS EmployeeDepart...	0 row(s) affected, 1 warning(s): 1305 PR
63	12:17:07	CREATE PROCEDURE EmployeeDepartmentDet...	0 row(s) affected
64	12:17:07	CALL EmployeeDepartmentDetails()	5 row(s) returned
65	12:17:39	DROP PROCEDURE IF EXISTS HighAverageDe...	0 row(s) affected, 1 warning(s): 1305 PR
66	12:17:39	CREATE PROCEDURE HighAverageDepartment...	0 row(s) affected
67	12:17:39	CALL HighAverageDepartments()	1 row(s) returned

7. Create a procedure using IF...ELSE to classify salary as High, Medium or Low.

Query 1 x

SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS SecondHighestSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE SecondHighestSalary()
6 BEGIN
7     SELECT emp_name, salary
8     FROM (
9         SELECT
10            emp_name,

```

Result Grid

emp_name	salary
Priya	67000.00

Result Grid

Form Editor

Result 19 x

Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
65	12:17:39	DROP PROCEDURE IF EXISTS HighAverageDe...	0 row(s) affected, 1 warning(s): 1305 PR
66	12:17:39	CREATE PROCEDURE HighAverageDepartment...	0 row(s) affected
67	12:17:39	CALL HighAverageDepartments()	1 row(s) returned
68	12:18:34	DROP PROCEDURE IF EXISTS SecondHighest...	0 row(s) affected, 1 warning(s): 1305 PR
69	12:18:35	CREATE PROCEDURE SecondHighestSalary() B...	0 row(s) affected
70	12:18:35	CALL SecondHighestSalary()	1 row(s) returned

8. Create a procedure that accepts a department ID and returns its average salary.

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS AboveDepartmentAverage
2
3 DELIMITER $$
4
5 CREATE PROCEDURE AboveDepartmentAverage()
6 BEGIN
7     SELECT e.emp_name,
8           e.salary,
9           d.dept_name
10    FROM employees e

```

Result Grid

	emp_name	salary	dept_name
▶	Anil	55000.00	CSE
	Sneha	70000.00	EEE

Export: Wrap Cell

Result Grid

Form Editor

Result 20 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
68	12:18:34	DROP PROCEDURE IF EXISTS SecondHighest...	0 row(s) affected, 1 warning(s): 1305 PR
69	12:18:35	CREATE PROCEDURE SecondHighestSalary() B...	0 row(s) affected
70	12:18:35	CALL SecondHighestSalary()	1 row(s) returned
71	12:19:14	DROP PROCEDURE IF EXISTS AboveDepartme...	0 row(s) affected, 1 warning(s): 1305 PR
72	12:19:14	CREATE PROCEDURE AboveDepartmentAvera...	0 row(s) affected
73	12:19:14	CALL AboveDepartmentAverage()	2 row(s) returned

9. Create a procedure using INOUT to apply a bonus to an input salary.

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS ApplyBonus;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE ApplyBonus(
6     INOUT salary_amount DECIMAL(10,2)
7 )
8 BEGIN
9     SET salary_amount = salary_amount + 5000;
10 END $$

```

Result Grid

UpdatedSalary
55000.00

Result Grid

Form Editor

Result 21 x Read Only Context Help Snip

Output

Action Output

#	Time	Action	Message
73	12:19:14	CALL AboveDepartmentAverage()	2 row(s) returned
74	12:19:56	DROP PROCEDURE IF EXISTS ApplyBonus	0 row(s) affected, 1 warning(s): 1305 PR
75	12:19:56	CREATE PROCEDURE ApplyBonus(INOUT s...	0 row(s) affected
76	12:19:56	SET @salary = 50000	0 row(s) affected
77	12:19:56	CALL ApplyBonus(@salary)	0 row(s) affected
78	12:19:56	SELECT @salary AS UpdatedSalary LIMIT 0, 1000	1 row(s) returned

10. Create a procedure that performs two related updates inside a transaction.

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DROP PROCEDURE IF EXISTS TransferSalary;
2
3 DELIMITER $$
4
5 CREATE PROCEDURE TransferSalary(
6     IN fromEmp INT,
7     IN toEmp INT,
8     IN amount DECIMAL(10,2)
9 )
10 BEGIN

```

Result Grid

	emp_id	emp_name	salary
▶	101	Rahul	51000.00
	103	Anil	57000.00
*	NULL	NULL	NULL

Form Editor

employees 22 x Apply Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 77	12:19:56	CALL ApplyBonus(@salary)	0 row(s) affected
✓ 78	12:19:56	SELECT @salary AS UpdatedSalary LIMIT 0, 1000	1 row(s) returned
⚠ 79	12:20:38	DROP PROCEDURE IF EXISTS TransferSalary	0 row(s) affected, 1 warning(s): 1305 PR
✓ 80	12:20:38	CREATE PROCEDURE TransferSalary(IN fro...	0 row(s) affected
✓ 81	12:20:38	CALL TransferSalary(101,103,2000)	0 row(s) affected
✓ 82	12:20:38	SELECT emp_id, emp_name, salary FROM emplo...	2 row(s) returned

7. Common Errors

Procedure already exists: Use DROP PROCEDURE IF EXISTS before recreating it.

Query 1 x

SQLAdditions

Limit to 1000 rows

```
1 CREATE PROCEDURE GetEmployees()  
2 BEGIN  
3 SELECT * FROM employees;  
4 END
```

Automatically disabled. manually get caret position and auto

Context Help Snip

Output

Action Output

#	Time	Action	Message
✓ 80	12:20:38	CREATE PROCEDURE TransferSalary(IN fro...	0 row(s) affected
✓ 81	12:20:38	CALL TransferSalary(101,103,2000)	0 row(s) affected
✓ 82	12:20:38	SELECT emp_id, emp_name, salary FROM emplo...	2 row(s) returned
✓ 83	12:22:27	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned
✓ 84	12:26:47	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
✗ 85	12:26:58	CREATE PROCEDURE GetEmployees() BEGIN ...	Error Code: 1064. You have an error in y

Syntax error near BEGIN: Check DELIMITER \$\$ and DELIMITER ;.

Query 1 x SQLAdditions

Limit to 1000 rows

```

1 DELIMITER $$
2
3 • CREATE PROCEDURE Example()
4 BEGIN
5     SELECT * FROM employees;
6 END $$
7
8 DELIMITER ;

```

Automatically disabled. manually get caret position and auto

Context Help Snip

Output

Action Output

#	Time	Action	Message
81	12:20:38	CALL TransferSalary(101,103,2000)	0 row(s) affected
82	12:20:38	SELECT emp_id, emp_name, salary FROM emplo...	2 row(s) returned
83	12:22:27	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned
84	12:26:47	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
85	12:26:58	CREATE PROCEDURE GetEmployees() BEGIN ...	Error Code: 1064. You have an error in y
86	12:27:23	CREATE PROCEDURE Example() BEGIN SEL...	0 row(s) affected

Table does not exist: Run USE employeedb1; and SHOW TABLES;.

Query 1 x

SQLAdditions

Limit to 1000 rows

1 • `SELECT * FROM employees;`

Result Grid

	emp_id	emp_name	dept_id	salary
▶	101	Rahul	1	51000.00
	102	Priya	3	67000.00
	103	Anil	1	57000.00
	104	Sneha	3	70000.00
	105	Kiran	2	48000.00
*	NULL	NULL	NULL	NULL

Form Editor

employees 25 x

Apply

Context Help

Output

Action Output

#	Time	Action	Message
✓ 84	12:26:47	DROP PROCEDURE IF EXISTS GetEmployees	0 row(s) affected
✗ 85	12:26:58	CREATE PROCEDURE GetEmployees() BEGIN ...	Error Code: 1064. You have an error in y
✓ 86	12:27:23	CREATE PROCEDURE Example() BEGIN SEL...	0 row(s) affected
✓ 87	12:27:44	USE employeeedb1	0 row(s) affected
✓ 88	12:27:44	SHOW TABLES	2 row(s) returned
✓ 89	12:27:53	SELECT * FROM employees LIMIT 0, 1000	5 row(s) returned

Unknown column: Check names using DESCRIBE employees;

